



FPT UNIVERSITY

Project Management Plan

Frevia

GroupBar	
Group Members	Pham Kha Vy - CE182254 Nguyen Tien Thuan - CE181024 Tran Phat Tai - CE181632 Vo Nhut Tin - CE180619 Nguyen Lu Nhat Huy - CE181196
Supervisor	Mrs. Cao Hong Nga
Ext Supervisor	
Capstone Project code	SEP490_04

Can Tho, September 2026

Contents

1	Overview	2
1.1	Scope & Estimation	2
1.2	Project Objectives	9
1.2.1	Key Performance Objectives	10
1.2.2	Testing Strategy & Quality Gates	10
1.3	Project Risks	10
2	Management Approach	12
2.1	Project Process	12
2.2	Quality Management	12
2.3	Training Plan	14
3	Project Organization	15
3.1	Team & Structure	15
3.2	Role & Responsibilities	15
4	Project Deliverables	15
5	Responsibility Assignments	16
6	Project Communications	18
7	Configuration Management	18
7.1	Document Management	18
7.2	Source Code Management	18
7.3	Tools & Infrastructures	19

1 Overview

1.1 Scope & Estimation

No	#	SMAR Item	Iteration Code	Complexity	Est. Effort (man days)
1		Authentication		–	–
2	1.1	Google Login	Sprint 1	Low	2
3	1.2	Login		Medium	2
4	1.3	Logout		Low	1
5		User Registration		–	–
6	1.4	Register	Sprint 1	Medium	2
7	1.5	Protected routes		Medium	2
8	1.6	Role-based access		Medium	3
9		Permission Management		–	–
10	1.7	Create Permission	Sprint 1	Medium	2
11	1.8	View Permissions List		Medium	1
12	1.9	Edit Permission		Medium	2
13	1.10	Delete Permission		Medium	1
14		User Management		–	–
15	1.11	Assign User Roles	Sprint 1	Medium	2
16	1.12	Avatar Cropping Tool		Low	2
17	1.13	Upload avatar		Medium	2
18	1.14	View Public Profile		Low	1
19	1.15	User Online Status		Medium	2
20		Job Management		–	–
21	1.16	Create job	Sprint 1	Medium	2
22	1.17	View job list		Low	1
23	1.18	Job listing UI		Medium	2
24	1.19	Job detail UI		Medium	2
25	1.20	Edit job / Delete job		Medium	2
26	1.21	Search jobs		Medium	2
27	1.22	Filter jobs		Medium	2
28	1.23	Job ownership validation		Low	1
29		Contract Management		–	–

No	#	SMAR Item	Iteration Code	Complexity	Est. Effort (man days)
30	1.24	Create Contract	Sprint 1	High	4
31	1.25	Update Contract		Medium	2
32	1.26	Complete Contract		Medium	2
33	1.27	Get Contract List		Low	1
34	1.28	Get Contract Detail		Low	1
35	1.29	Sign Contract		High	3
36	1.30	Cancel Contract		Medium	2
37		Contract Files	Sprint 1	–	–
38	1.31	Contract Files		Low	1
39	1.32	Upload Contract File		Medium	2
40	1.33	Delete Contract File		Low	1
37		Account Security	Sprint 2	–	–
38	2.1	Email Verification		Medium	2
39	2.2	Forgot Password		Medium	2
40	2.3	Reset Password		Medium	2
41	2.4	Session Management		Medium	2
42	2.5	Phone Number Verification		Medium	2
43	2.6	Resend OTP		Low	1
44	2.7	Change Password		Low	1
45	2.8	Email Notification Preferences		Low	1
46	2.9	Account Security Logs		Medium	2
47		Profile Management	Sprint 2	–	–
48	2.10	Edit client profile		Medium	2
49	2.11	Edit freelancer profile		Medium	2
50	2.12	Education & Certifications		Low	2
51	2.13	Language Proficiency		Low	1
52	2.14	Add freelancer skills		Low	1
53	2.15	Edit/Delete freelancer skills		Low	1
54	2.16	Add portfolio		Low	2
55	2.17	Edit/Delete portfolio		Low	1
56	2.18	Portfolio Media Upload		Medium	2
57	2.19	Social Link Management		Low	1

No	#	SMAR Item	Iteration Code	Complexity	Est. Effort (man days)
58		Job Management		–	–
59	2.20	Job status management		Medium	2
60	2.21	Job Draft	Sprint 2	Low	1
61	2.22	Re-open Job		Low	1
62	2.23	Sort Jobs		Low	1
63		Milestone Management		–	–
64	2.24	View Milestones List		Low	1
65	2.25	View Milestone Detail		Low	1
66	2.26	Add Milestone		Medium	2
67	2.27	Edit Milestone	Sprint 2	Medium	2
68	2.28	Delete Milestone		Low	1
69	2.29	Request Changes		Medium	2
70	2.30	Approve Milestone		Medium	2
71	2.31	Progress Milestone		Medium	2
72		Milestone Files		–	–
73	2.32	View Milestone Files		Low	1
74	2.33	Upload Milestone File	Sprint 2	Medium	2
75	2.34	Delete Milestone File		Low	1
76		Submission Management		–	–
77	2.35	Submit Milestone		Medium	2
78	2.36	Delete Submission	Sprint 2	Low	1
79	2.37	View Submission		Low	1
80	2.38	View Submission Detail		Low	1
69		AI Features		–	–
70	2.29	AI freelancer recommendation		High	4
71	2.30	Match score calculation		High	4
72	2.31	Recommended freelancer list	Sprint 2	Medium	2
73	2.32	AI Job Recommendation		High	4
74	2.33	AI Proposal Assistant		High	4
75	2.34	AI Sensitive Word Detection		High	4
76		Account Management		–	–
77	3.1	Account Deactivation		Low	2

No	#	SMAR Item	Iteration Code	Complexity	Est. Effort (man days)
78	3.2	OAuth Account Linking		Medium	2
79	3.3	Unlink OAuth Account		Medium	2
80	3.4	Linked Accounts UI		Low	1
81		Admin: Client Management	Sprint 3	–	–
82	3.5	Create client (Admin)		Low	2
83	3.6	View client list (Admin)		Medium	2
84	3.7	View client detail (Admin)		Low	1
85	3.8	Edit client (Admin)		Medium	2
86	3.9	Delete client (Admin)		Medium	2
87		Profile Extras	Sprint 3	–	–
88	3.10	Skill Endorsement		Low	2
89	3.11	Cover Image Upload		Low	1
90	3.12	Profile Completion Progress		Low	2
91	3.13	Upload ID Documents		Low	2
92	3.14	Admin Review ID Verification		Medium	2
93	3.15	ID Verified Badge Display		Low	1
94		Proposal Management	Sprint 3	–	–
95	3.16	Submit proposal		Medium	2
96	3.17	Edit proposal		Medium	2
97	3.18	Withdraw proposal		Low	1
98	3.19	View submitted proposals		Low	1
99	3.20	Proposal status tracking		Low	2
100	3.21	View proposal list (Client)		Low	1
101	3.22	View proposal detail (Client)		Low	1
102	3.23	Accept proposal		Medium	2
103	3.24	Reject proposal		Low	1
117		Review & Rating	Sprint 3	–	–
118	3.36	Add freelancer review		Medium	2
119	3.37	Add client review		Medium	2
120	3.38	Edit review		Low	1
121	3.39	Delete review		Low	1
122	3.40	View review list		Low	1

No	#	SMAR Item	Iteration Code	Complexity	Est. Effort (man days)
123	3.41	Calculate average rating		Low	2
124	3.42	Review authorization		Low	1
125		Dashboard & Notifications		–	–
126	3.43	View freelancer dashboard		Medium	2
127	3.44	View client dashboard	Sprint 3	Medium	2
128	3.45	In-app Notification Preferences		Low	1
129		Admin: Freelancer Management		–	–
130	4.1	Create freelancer (Admin)		Medium	2
131	4.2	View freelancer list (Admin)		Medium	2
132	4.3	View freelancer detail (Admin)		Low	1
133	4.4	Edit freelancer (Admin)	Sprint 4	Medium	2
134	4.5	Delete freelancer (Admin)		Medium	2
135	4.6	Admin Moderation Queue		Medium	2
136	4.7	User Activity Log		Medium	2
137		Freelancer Discovery		–	–
138	4.8	Search freelancer		Medium	2
139	4.9	Filter freelancer		Medium	2
140	4.10	Sort Freelancer List		Low	1
141	4.11	Ranking freelancer		Medium	2
142	4.12	Saved Search	Sprint 4	Low	2
143	4.13	Recently Viewed Jobs		Low	1
144	4.14	Freelancer Availability Status		Low	1
145	4.15	Portfolio Like & Save		Low	2
146	4.16	Profile Visit Analytics		Low	2
147		Social & Bookmarks		–	–
148	4.17	Follow Freelancer		Medium	2
149	4.18	Favorite Freelancer List		Low	1
150	4.19	Save / Bookmark Job	Sprint 4	Low	1
151	4.20	Job Alert		Medium	2
152		Proposal Enhancements		–	–
153	4.21	Proposal Template		Low	2
154	4.22	Proposal Archive	Sprint 4	Low	1

No	#	SMAR Item	Iteration Code	Complexity	Est. Effort (man days)
155	4.23	Attach File to Proposal		Low	2
156	4.24	Proposal Auto Save Draft		Medium	2
157		Collaboration Tools		–	–
158	4.25	Direct Messaging (Basic)		High	4
159	4.26	Task Progress Tracking		Medium	2
160	4.27	Shared File Upload		Medium	2
161	4.28	Task Checklist	Sprint 4	Low	2
162	4.29	Add Task Checklist Item		Low	1
163	4.30	Edit / Delete Task Item		Low	1
164	4.31	Mark Task Complete		Low	1
174		Delivery & Review		–	–
175	4.40	Accept / Reject Final Delivery		Medium	2
176	4.41	Review Response	Sprint 4	Low	1
177	4.42	Breakdown Rating		Medium	2
178		Analytics & Admin		–	–
179	4.43	Advanced Analytics Dashboard		Medium	4
180	4.44	Export Reports (CSV/PDF)		Low	2
181	4.45	Multi-language Support (i18n)	Sprint 4	Medium	4
182	4.46	Admin Panel (MVP)		Medium	4
183		Admin: Content & Settings		–	–
184	5.1	Banned Keyword Management		Low	2
185	5.2	Static Content Management		Low	2
186	5.3	System Overview Dashboard		Medium	2
187	5.4	Enable TOTP 2FA		Medium	2
188	5.5	TOTP Challenge on Login	Sprint 5	Medium	2
189	5.6	Disable TOTP 2FA		Low	1
190	5.7	General Platform Settings		Medium	2
191	5.8	Email & SMTP Configuration		Medium	2
192	5.9	Job Automation Defaults		Low	2
193		Forum Management		–	–
194	5.10	Create forum post		Medium	2
195	5.11	View forum posts		Low	1

No	#	SMAR Item	Iteration Code	Complexity	Est. Effort (man days)
196	5.12	View forum post detail		Low	1
197	5.13	Edit forum post		Low	1
198	5.14	Delete forum post		Low	1
199	5.15	Add forum comment		Low	1
200	5.16	Edit/Delete forum comment		Low	1
201	5.17	Like forum post		Low	1
202	5.18	Search forum posts		Low	2
203	5.19	Report forum content		Low	1
204		Job & Proposal Enhancements	Sprint 5	–	–
205	5.20	Recent Job Search History		Low	1
206	5.21	Proposal Deadline Validation		Low	1
207	5.22	Client Proposal Comparison		Medium	2
208	5.23	Proposal Notification		Medium	2
209	5.24	Job Expiration System		Medium	2
210	5.25	Sort proposal list		Low	1
211		Admin: Job Management	Sprint 5	–	–
212	5.26	View All Jobs (Admin)		Medium	2
213	5.27	Set Featured Job Manually		Low	1
214	5.28	Extend Job Posting Duration		Low	1
215		Dispute Management	Sprint 3	–	–
216	3.1	Open Dispute		Medium	2
217	3.2	View Dispute Detail		Low	1
218	3.3	View Dispute List		Low	1
219	3.4	Respond to Dispute		Medium	2
220	3.5	Respond to Dispute Resolution		Medium	2
221	3.6	Propose Dispute Resolution		High	3
222		Admin Contract Dispute Management	Sprint 3	–	–
223	3.7	Admin Cancel Contract		High	3
224	3.8	Admin Update Contract Status		Medium	2
225	3.9	View Contract Disputes		Low	1
222		Admin: Contract Management		–	–

No	#	SMAR Item	Iteration Code	Complexity	Est. Effort (man days)
223	5.35	View All Contracts (Admin)	Sprint 5	Medium	2
224	5.36	View Contract Detail (Admin)		Low	1
225	5.37	Admin Cancel Contract (Force)		High	4
226	5.38	Contract Audit Dashboard		Medium	2
227		Payment	Sprint 4	–	–
228	4.1	Stripe Connect Account		High	4
229	4.2	Payout History		Medium	2
230	4.3	Payout Detail		Low	1
231	4.4	Create Payment Intent		High	3
232	4.5	Refund Payment		High	3
233	4.6	Payment History		Low	1
234	4.7	Payment Detail		Low	1
235	4.8	Release Payment		High	3
236	4.9	Execute Dispute Resolution Payment		High	4
232		Notifications & Announcements	Sprint 5	–	–
233	5.43	Real-time Notification Center		High	4
234	5.44	System Announcement Banner		Low	1
235		Forum Categories	Sprint 5	–	–
236	5.45	Create forum category		Low	1
237	5.46	View forum category list		Low	1
238	5.47	Edit forum category		Low	1
239	5.48	Delete forum category		Low	1
240		Admin: Moderation & Reports	Sprint 5	–	–
241	5.49	View Hidden / Reported Proposals		Low	1
242	5.50	Restore Removed Jobs		Low	1
243	5.51	Admin Action Log		Medium	2
244	5.52	Scheduled Report Export		Medium	2
245	5.53	Advertisement Banner Management		Low	2

1.2 Project Objectives

The **Frevia** (codename GFY) project develops a freelance marketplace platform connecting Clients and Freelancers in areas such as design, programming, editing, consulting, etc. The system allows Clients to post jobs, Freelancers

to submit proposals, create contracts, manage milestones, process secure Stripe payments, write reviews, and includes AI support for recommendations and sensitive-content detection. Additionally, the platform integrates a community forum, collaboration tools (chat, file sharing, checklists), analytics dashboards, and a comprehensive administration system. Based on the established backlog of **207 functional tasks** (MVP + extended sprints), the project is divided into 5 sprints with a total estimated effort of **372 man-days** (see the *Scope and Estimation* table).

1.2.1 Key Performance Objectives

- **Timeliness (%):** Deliver all sprints and milestones on schedule.
Target: 95% of the tasks (i.e., 197 out of 207) must be completed on time per sprint plan. Delayed tasks shall not exceed 5% of the total.
- **Allocated Effort (man-days):** Optimise development resources, aligning effort with task complexity (Low/Medium/High).
Target: Actual total man-days shall not exceed $372 \pm 5\%$ (max. 391 man-days). High-complexity tasks (AI, Stripe payment integration, dispute resolution) are prioritised for experienced team members.
- **Defect Distribution:** Maintain high software quality throughout development, testing, and production.
Target:
 - No critical defects in the production environment after launch.
 - 95% of high-priority defects fixed within 48 hours of detection.
 - Medium-priority defects not exceeding 5% of the total system defects.

1.2.2 Testing Strategy & Quality Gates

All 207 tasks must pass through the following testing stages. The table below summarises test coverage targets and defect resolution KPIs.

#	Testing Stage	Test Coverage (%)	No. of Defects	% of Defect Solved
1	Reviewing (Code review & design review)	100	0	100
2	Unit Test	100	0	100
3	Integration Test	100	0	100
4	System Test	100	0	100
5	Acceptance Test	95	0	100

- Reviewing includes peer review, static analysis for backend (NestJS) and frontend (NextJS), as well as Stripe API and webhook integration review.
- Unit Test requires at least 80% coverage for critical modules (authentication, job, proposal, contract, and payment).
- Integration Test verifies main flows: create job → submit proposal → accept → create Stripe PaymentIntent → create milestone → confirm or transfer payment.
- System Test executes end-to-end scenarios on a staging environment using Stripe test mode and webhook events.
- Acceptance Test is performed with at least 3 Clients and 5 Freelancers, ensuring 95% of key business work-flows function correctly.

1.3 Project Risks

The following risks have been identified based on the project backlog and technical complexity (Stripe integration, AI modules, distributed system, etc.). Mitigation strategies are defined to ensure timely delivery and quality.

Risk Description	Impact	Possibility	Response Plans
Stripe Payment Integration Failures (PaymentIntent, webhook, milestone payment, and refund processing)	High	High	Use Stripe test mode and automated integration tests. Verify webhook signatures, make handlers idempotent, and reconcile payment status from Stripe before updating local records. Define retry and refund procedures.
AI Model Integration Failure (recommendation engine, sensitive word detection, proposal assistant)	High	High	Build fallback rule-based models. Validate AI outputs with human review in early sprints. Use A/B testing. Prepare mock APIs to avoid blocking frontend development.
Payment Failure and Dispute Management (card declines, payment disputes, refunds, or delayed webhook events)	Medium	High	Display clear payment status and error messages. Support payment retries and refunds, log all Stripe events, and provide an admin workflow for reviewing disputes and reconciliation differences.
Scope Creep from 207 Tasks (many features across 5 sprints)	High	High	Strictly follow the backlog prioritisation (MVP first). Use change control board for any addition. Reject non-critical features beyond planned man-days (532 $\pm$ 5%).
Dispute Resolution Delays (admin handling payment disputes or milestone disagreements may be slow)	Medium	Medium	Define SLAs for dispute processing (max 48 hours). Automate evidence collection. Train at least 3 admins. Implement escalation path.
Real-time Notification System Overload (high concurrency, WebSocket)	Medium	Medium	Use message queue (Redis/RabbitMQ). Design idempotent consumers. Horizontal scaling for notification service. Perform load testing before sprint 5.
Third-party API Availability (Google OAuth, email SMTP, cloud storage)	Medium	Low	Implement circuit breakers and retry logic. Cache frequently used data. Have fallback authentication (email/password) always available.
Team Knowledge Gap in Stripe Integration (only a few members experienced with payment APIs and webhooks)	High	Medium	Conduct internal Stripe training before sprint 3. Use official Stripe documentation, test-mode fixtures, pair programming, and code review for payment and webhook flows.
Data Privacy Compliance (GDPR, local laws for personal data)	Medium	Medium	Anonymise user data in logs. Provide data export/deletion endpoints. Add privacy policy and consent checkboxes. Regular legal review.
Integration Testing Bottleneck (207 tasks, many cross-module dependencies)	Medium	High	Automate integration tests in CI/CD pipeline. Use contract testing (Pact) between frontend/backend. Allocate dedicated testing environment.

2 Management Approach

The Freelance Marketplace Platform project adopts the Scrum framework as its primary management approach. Scrum provides an iterative and incremental development process that enables the team to continuously deliver valuable features while adapting to evolving project requirements. This approach promotes collaboration, transparency, and continuous improvement throughout the software development lifecycle.

The project team consists of members responsible for system analysis, UI/UX design, frontend development, backend development, testing, and project management. Through regular communication and collaboration, the team works together to ensure that project objectives are achieved efficiently and that the platform meets stakeholder and user expectations.

The platform aims to connect clients and freelancers through a secure digital marketplace where clients can post projects, hire suitable freelancers, and manage transactions through a Stripe-based payment mechanism. Stripe PaymentIntents, webhooks, refunds, and transfers are used to support milestone payments and payment-status tracking. To support this vision, project requirements are organized into a Product Backlog and prioritized according to business value and development priorities.

The development process is divided into five sprints over a three-month period. Each sprint focuses on delivering incremental improvements and essential functionalities, allowing the team to gather feedback, evaluate progress, and continuously refine the product. Core features include user management, project posting, proposal submission, contract management, communication tools, secure payment processing, and administrative functions.

By following Scrum principles, the project maintains flexibility, encourages frequent stakeholder feedback, and ensures the continuous delivery of a reliable and user-centered Freelance Marketplace Platform.

2.1 Project Process

The Freelance Marketplace Platform project follows the Scrum framework and is developed over a period of three months, divided into five sprints. Each sprint focuses on delivering a specific set of functionalities and improvements, ensuring continuous progress and alignment with project objectives.

During each sprint, the team conducts the following Scrum events:

- **Sprint Planning Meeting:** Defines sprint goals, selects Product Backlog items, and creates a development plan for the upcoming sprint.
- **Daily Stand-up Meeting:** A short daily meeting where team members discuss completed work, current progress, challenges, and planned tasks.
- **Sprint Review Meeting:** Demonstrates completed features, evaluates sprint outcomes, and gathers feedback from supervisors and stakeholders.
- **Sprint Retrospective Meeting:** Reviews the team's performance, identifies areas for improvement, and proposes actions to enhance future sprints.

Throughout the development process, the Product Backlog is continuously refined and prioritized to reflect changing requirements and stakeholder feedback. Team members collaborate closely to complete assigned tasks and ensure that implemented features meet the expected quality standards.

At the end of each sprint, completed functionalities are tested and evaluated to verify that they satisfy both functional and non-functional requirements. Feedback collected during Sprint Review meetings is incorporated into future sprint planning, while insights gained from Sprint Retrospective meetings help improve development practices and team efficiency.

By following a structured Scrum process with five sprints, the project maintains flexibility, encourages continuous feedback, and ensures the gradual delivery of a secure, reliable, and user-centered Freelance Marketplace Platform.

2.2 Quality Management

To ensure the successful development and delivery of the Freelance Marketplace Platform, a comprehensive quality management strategy is applied throughout the entire software development lifecycle. The primary objective is to deliver a secure, reliable, scalable, and user-friendly platform that effectively connects clients and freelancers while maintaining high standards of software quality.

- **Defect Prevention**
 - Conducting detailed requirement analysis and clarification sessions to reduce misunderstandings and incomplete specifications.

- Defining coding standards, naming conventions, and development guidelines to ensure consistency across the project.
- Applying software engineering best practices during design and implementation to minimize potential defects.
- **Review Process**
 - Performing regular code reviews to improve code quality, maintainability, and compliance with project standards.
 - Reviewing system architecture, database design, API specifications, and project documentation before implementation.
 - Encouraging peer feedback and collaborative problem-solving throughout the development process.
- **Unit Testing**
 - Developing unit tests for critical business logic and core modules.
 - Verifying the correctness of individual components before integration.
 - Refactoring code when necessary while ensuring all tests continue to pass successfully.
- **Integration Testing**
 - Testing interactions between frontend, backend services, databases, and third-party integrations.
 - Validating data flow and communication among system components.
 - Identifying and resolving integration issues before system deployment.
- **System Testing**
 - Conducting end-to-end testing to verify the complete functionality of the platform.
 - Testing major features such as user registration, authentication, project posting, proposal submission, contract management, messaging, notifications, and payment workflows.
 - Ensuring compliance with both functional and non-functional requirements.
- **User Acceptance Testing (UAT)**
 - Validating the system against real-world business scenarios and user expectations.
 - Testing workflows involving clients, freelancers, and administrators.
 - Collecting feedback from supervisors, stakeholders, and potential users before final release.
- **Performance Testing**
 - Evaluating system responsiveness under different workloads and usage conditions.
 - Measuring response times for project search, freelancer search, messaging, and transaction processing.
 - Identifying and resolving performance bottlenecks to ensure a smooth user experience.
- **Security and Data Protection**
 - Implementing authentication and role-based authorization mechanisms.
 - Protecting sensitive user and transaction data through secure storage and transmission practices.
 - Preventing common web vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), and unauthorized access attempts.
 - Conducting security testing to identify and mitigate potential risks.
- **Transaction and Payment Validation**
 - Verifying the accuracy and consistency of payment-related operations.
 - Testing Stripe PaymentIntent workflows, webhook-driven status updates, refunds, and payment records.
 - Ensuring transaction integrity, idempotent webhook processing, and preventing duplicate or invalid payments.
 - Maintaining accurate financial records throughout the platform.
- **Continuous Monitoring and Feedback**

- Tracking defects, enhancement requests, and system issues throughout development.
- Collecting feedback from supervisors, stakeholders, and test users.
- Continuously improving the platform based on testing outcomes and user feedback.

- **Scrum Process Integration**

- Integrating quality assurance activities into every sprint.
- Conducting Sprint Planning meetings to establish quality objectives and deliverables.
- Using Daily Stand-up meetings to identify and address issues promptly.
- Reviewing completed features during Sprint Reviews and discussing improvements during Sprint Retrospectives.

- **Quality Metrics and Reporting**

- Monitoring key quality indicators such as defect count, bug resolution time, test coverage, feature completion rate, and system response time.
- Maintaining issue-tracking records and quality reports throughout the project lifecycle.
- Using collected metrics to support continuous quality improvement and project evaluation.

- **Documentation Management**

- Maintaining comprehensive and up-to-date technical documentation.
- Documenting system architecture, database schemas, API specifications, and development decisions.
- Ensuring documentation remains accessible and understandable for all project members.
- Supporting future maintenance, scalability, and knowledge transfer through proper documentation practices.

By implementing these quality management practices, the project ensures the development of a high-quality Freelance Marketplace Platform that delivers a secure, efficient, and reliable environment for clients and freelancers while supporting future growth and maintainability.

2.3 Training Plan

Training Area	Participants	When, Duration	Waiver Criteria
GitHub Workflow & Version Control	x	Week 1 (2 days)	Mandatory
Next.js Web Development	x	Week 1 (1 week)	Mandatory
React Native Mobile Development (or Flutter)	x	Week 1-2 (2 weeks)	Mandatory
NestJS Backend Development	x	Week 1-2 (2 weeks)	Mandatory
PostgreSQL Database Design and Management	x	Week 2 (3 days)	Mandatory
RESTful API Design & Swagger Documentation	x	Week 3 (3 days)	Mandatory
Real-time Communication (Socket.IO)	x	Week 3 (3 days)	Mandatory
Stripe Payment Integration (Checkout, PaymentIntent, Webhooks)	x	Week 4 (1 week)	Mandatory

Continued on next page

Training Area	Participants	When, Duration	Waiver Criteria
Docker Fundamentals	x	Week 4 (3 days)	Mandatory
CI/CD with GitHub Action	x	Week 5 (3 days)	Mandatory
UI/UX Design with Figma	x	Project Start (3 days)	Mandatory

3 Project Organization

3.1 Team & Structure

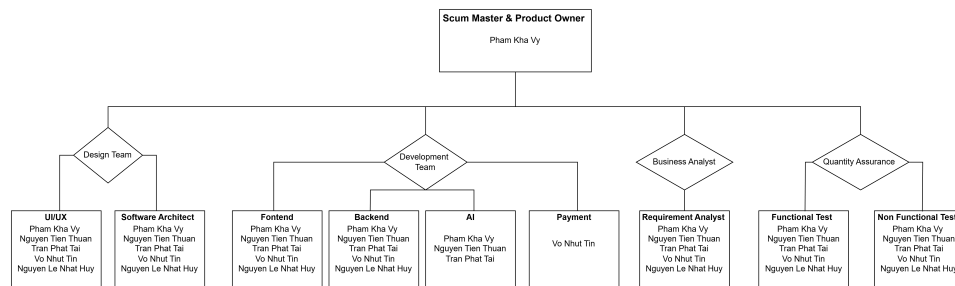


Figure 1: Team and Structure Diagram

3.2 Role & Responsibilities

4 Project Deliverables

#	Deliverable	Duration	Start	Finish	Sprint	Week
1	Product Introduction Document	7 days			Sprint 1	Week 1
2	Project Management Plan Document	7 days			Sprint 1	Week 2
3	Software Requirements Specification Document	7 days			Sprint 1	Week 3
4	Software Design Document	7 days			Sprint 1	Week 4
5	Implementation	56 days			Sprint 2–4	Week 5–12
6	Code Iteration 1	14 days			Sprint 2	Week 5–6
7	Update SRS & SDS (Iteration 1)	2 days				
8	Create Test Case (Iteration 1)	5 days				
9	Stripe Payment and Contract Workflow	10 days				
10	Frontend Contract Management UI	10 days				
11	Stripe Checkout / PaymentIntent Integration	5 days				
12	Code Iteration 2	14 days			Sprint 3	Week 7–8
13	Update SRS & SDS (Iteration 2)	2 days				
14	Create Test Case (Iteration 2)	5 days				
15	Milestone Payment Logic	10 days				
16	Freelancer Dashboard	10 days				
17	Client Dashboard	10 days				
18	Code Iteration 3	14 days			Sprint 4	Week 9–10
19	Update SRS & SDS (Iteration 3)	2 days				
20	Create Test Case (Iteration 3)	5 days				
21	AI Freelancer Matching Module	10 days				

#	Deliverable	Duration	Start	Finish	Sprint	Week
22	Stripe Payment History and Reconciliation	10 days				
23	Stripe Payment and Webhook Security Testing	10 days				
24	Code Iteration 4	14 days			Sprint 4	Week 11–12
25	Update SRS & SDS (Iteration 4)	2 days				
26	Create Test Case (Iteration 4)	5 days				
27	UAT Testing	7 days				
28	Bug Fixing	7 days				
29	Verification Software	5 days			Sprint 4	Week 13
30	Update Document & User Guide	5 days				
31	Software Optimization	3 days			Sprint 4	Week 14
32	Execute UAT & Fix Bugs	3 days				
33	Deliver Report (Final Project Report)	1 day			Sprint 4	Week 14
34	Thesis Presentation	13 days			Sprint 4	Week 15–16

Table 5: Project Deliverables

5 Responsibility Assignments

R = Responsible; A = Accountable; C = Consulted; I = Informed

Responsibility	VyPK	ThuanNT	TaiNP	TinVN	HuyNLN
Project initiation & scope definition	RA	CI	CI	CI	CI
Project risk definition & planning	RA	CI	CI	CI	CI
Project Introduction Document	RA	CI	CI	CI	CI
Project Management Plan	RA	CI	CI	CI	CI
Requirements gathering & User Stories	A	R	R	CI	CI
UI/UX Prototype	A	R	R	CI	CI
Software Architecture Design	A	R	R	R	R
Stripe Payment Integration	A	CI	CI	R	CI
Stripe API and Webhook Integration	A	CI	CI	R	CI
Stripe Checkout / PaymentIntent Integration	A	CI	CI	R	CI
Frontend Development	A	R	R	CI	R
Backend Development	A	R	R	CI	R
AI Matching Module	A	R	R	CI	R
Integration Testing	A	CI	R	CI	R
Unit Testing	A	R	C	R	C
Payment and Webhook Security Testing	A	CI	C	R	CI
User Acceptance Testing	RA	I	I	I	I
Technical Training	RA	R	R	R	R
Final Report	RA	R	R	R	R
Thesis Presentation	RA	R	R	R	R

Table 6: Responsibility Assignments Matrix

Role Name	Description
Product Owner	The visionary leader responsible for the product's direction and backlog prioritization, ensuring the product meets customer needs.
Scrum Master	The facilitator and coach, ensuring the Scrum process runs smoothly and the team follows Agile practices.
UX/UI Designer	Combines user research with visual design to create interfaces that are intuitive, user-friendly, and visually appealing.
Software Architect	Technical visionary responsible for defining the system architecture, technology stack, and development standards.
Frontend Developer	Develops responsive and interactive user interfaces, translating design mockups into functional web pages and ensuring a seamless user experience.
Backend Developer	Implements server-side logic, APIs, database management, authentication, and business processes that support application functionality.
AI Developer	Designs, develops, and trains intelligent models and algorithms to provide AI-powered features and automated decision-making capabilities.
Payment Integration Developer	Develops and maintains Stripe Checkout or PaymentIntent flows, webhook handling, payment status synchronization, refunds, transfers, and transaction reconciliation.
Requirement Analyst	Acts as the bridge between stakeholders and the development team by gathering, analyzing, documenting, and validating project requirements.
Functional Tester	Ensures that all system features and functions operate according to the specified requirements through systematic testing and defect verification.
Non-Functional Tester	Evaluates system quality attributes such as performance, scalability, security, reliability, and usability to ensure the application meets quality standards.

Table 4: Roles and Responsibilities

6 Project Communications

Effective communication is essential to ensure successful collaboration among team members, supervisors, and stakeholders throughout the project lifecycle.

Communication Activity	Participants	Frequency	Purpose
Daily Stand-up	Development Team	Daily	Discuss progress, blockers and planned tasks
Sprint Planning	Team Members, Supervisor	Each Sprint	Define sprint goals and backlog items
Sprint Review	Team Members, Supervisor	Each Sprint	Review completed features and collect feedback
Sprint Retrospective	Development Team	Each Sprint	Identify improvement opportunities
Progress Report	Project Leader, Supervisor	Weekly	Track project status and risks
Technical Discussion	Development Team	As Needed	Resolve technical issues and design decisions

Table 7: Project Communication Plan

Communication is conducted through Google Meet, Discord, GitHub, and Google Drive. Important decisions and meeting outcomes are documented and shared with all relevant stakeholders.

7 Configuration Management

7.1 Document Management

All project documents are maintained in a centralized repository using Google Drive and GitHub.

The managed documents include:

- Project Management Plan (PMP)
- Software Requirements Specification (SRS)
- Software Design Specification (SDS)
- Test Cases and Test Reports
- User Manuals
- Meeting Minutes

Document versions are tracked using version numbers and update dates. Major modifications require review and approval before publication.

Obsolete versions are archived to maintain traceability and historical records.

7.2 Source Code Management

The project uses Git and GitHub as the primary source code management platform.

The following branching strategy is applied:

- main: stable production-ready code
- develop: integration branch
- feature/*: feature development branches
- hotfix/*: urgent bug fixes

All code changes are committed using meaningful commit messages and reviewed through Pull Requests before merging into the main branch.

Code reviews help ensure code quality, maintainability, and consistency throughout the project.

7.3 Tools & Infrastructures

Category	Tool
Project Management	Jira / Trello
Version Control	GitHub
Communication	Discord, Google Meet
Documentation	Google Drive, Microsoft Word
UI/UX Design	Figma
Frontend Development	Next.js, React
Backend Development	NestJS
Database	PostgreSQL
Payment Integration	Stripe Checkout, PaymentIntents, Webhooks, Stripe CLI
API Documentation	Swagger
Testing	Jest, Postman
Containerization	Docker
CI/CD	GitHub Actions

Table 8: Tools and Infrastructures